Mastering Claude Code: The Complete Architecture-Informed Guide

> Everything a software engineer needs to know, grounded in how Claude Code actually works under the hood.

---

1. Understand What You're Actually Working With

Claude Code is **not** an autocomplete tool or a chat wrapper. It's a single-threaded agentic loop (internally codenamed "nO") that calls the Claude model repeatedly, executing tools on your local machine between each call. Every time the model returns `tool_use` blocks, the loop executes them (file reads, bash commands, edits) and feeds results back. When the model returns plain text with no tool calls, the loop ends and control returns to you.

The key mental model: Your prompt doesn't just generate code — it launches an autonomous agent that will read, think, act, verify, and iterate. The quality of that entire multi-step loop depends on how well you set up its context, how clearly you define the task, and how effectively you constrain the action space.

What happens on every API call: Claude Code assembles a prompt from your system prompt, CLAUDE.md contents, tool definitions (including any MCP servers), conversation history, and your latest message. This goes to the Anthropic API where it still passes through auth, input moderation, inference, output moderation, and streaming — Claude Code doesn't bypass those safety layers. But Claude Code *controls* the system prompt, tool orchestration, and context management layers, running them locally instead of server-side.

The tools Claude has access to:

- **Read tools:** Read, Glob, Grep, LS, NotebookRead — for understanding your codebase
- **Write tools:** Edit, MultiEdit, Write, NotebookEdit — for making changes
- **Execution:** Bash (persistent shell sessions) — for running anything: tests, builds, git, docker
- **Network:** WebFetch, WebSearch — for documentation and research
- **Planning:** TodoRead, TodoWrite — for structured task tracking
- **Delegation:** Task (subagents) — for parallel or isolated work

Understanding this architecture explains *why* every best practice works. Nothing in this guide is arbitrary — it all flows from the mechanics of the system.

---

2. CLAUDE.md Is Your Highest-Leverage Investment

Why it matters architecturally

CLAUDE.md is loaded into **every single session** as part of the system prompt. It's the only persistent memory Claude Code has between sessions. The model is stateless — its weights are frozen, it learns nothing between

conversations. CLAUDE.md is how you bridge that gap.

Think of it as onboarding documentation for a brilliant but amnesiac contractor who joins your team fresh every morning. They can learn fast, but they start from zero every time.

What to include: The WHAT, WHY, HOW framework

WHAT — Give Claude a map of your codebase:

- Project structure and tech stack
- Key directories and what they contain
- Monorepo layout (which apps, shared packages, what everything is for)
- Database schema locations and ORM patterns

WHY — Explain the purpose and rationale:

- Purpose of each major module
- Architectural decisions and *why* they were made (not just what they are)
- Design patterns in use and why they were chosen
- Known quirks, gotchas, or legacy decisions

HOW — Tell Claude how to work on the project:

- Build, test, lint, typecheck commands
- How to verify changes (which test suite, which linting config)
- Package manager (bun vs npm vs pnpm)
- Branch naming, commit conventions, PR process
- Environment setup notes (pyenv, nvm, docker requirements)

What NOT to include

Don't include things Claude already does correctly. If Claude never makes a specific mistake in your codebase, the rule is wasted context. Only document what Claude gets wrong.

Don't include rare or situation-specific instructions. Move these to Skills (which load on demand, saving context in every other session). CLAUDE.md is for universally-applicable rules only.

Don't write negative-only rules. "Never use CommonJS" leaves Claude stuck. Always provide the alternative: "Never use CommonJS; always use ES modules with `import`/`export`."

Don't embed large reference docs. Instead, reference file paths: "For MUI component patterns, see `src/components/README.md`." Claude will read the file only when needed, saving context when it doesn't.

Don't use `@-file` directives to embed docs. These embed the entire file on every run. Instead, point Claude to the path and let it read on demand.

Keep it lean — this is critical

Research suggests frontier thinking models can reliably follow approximately 150-200 instructions. Claude Code's own system prompt already consumes about 50 of those instruction "slots." An overloaded CLAUDE.md means critical rules get lost in the noise.

Target: under ~500 lines. Ruthlessly prune. Version control it. Review it like you'd review code — because it *is* code that shapes your AI's behavior.

Use hierarchical CLAUDE.md files

Put general project rules in the root. Add specific rules in subdirectories (`/frontend/CLAUDE.md`, `/backend/CLAUDE.md`). Claude prioritizes the most specific (most nested) file when working in that area. This is especially powerful in monorepos.

Iterate continuously

After sessions where Claude makes mistakes, add rules to prevent those specific mistakes. After sessions where Claude follows a rule unnecessarily, remove it. Your CLAUDE.md should evolve like a living document — reflecting your team's actual experience.

---

3. Plan Before You Build — Every Time

Why planning matters architecturally

Each step in an agentic chain introduces error probability. If each step is 95% accurate, 10 steps give you only ~60% end-to-end accuracy. Planning reduces the total number of steps by getting direction right upfront and reduces per-step error by giving each step clearer context.

Without planning, Claude will start coding immediately and often build the wrong thing confidently. With planning, Claude asks the right questions, considers edge cases, and produces code that's right the first time far more often.

The interview pattern (for larger features)

```
I want to build [brief description]. Interview me in detail using
the AskUserQuestion tool. Ask about technical implementation, UI/UX,
edge cases, concerns, and tradeoffs. Don't ask obvious questions —
dig into the hard parts I might not have considered. Keep interviewing
until we've covered everything, then write a complete spec to SPEC.md.
```

Then start a **fresh session** (`/clear`) to implement the spec. The new session has clean context focused entirely on implementation, and a written spec it can read.

This pattern works because it forces requirements to be explicit *before* Claude commits to an implementation direction. It also surfaces assumptions you didn't know you were making.

The spec-then-execute pattern

1. Session 1: Research, plan, write to `PLAN.md` or `SPEC.md`
2. You review and refine the plan
3. Session 2 (fresh): "Implement the plan in PLAN.md step by step"

Each session starts with full context budget, clean history, and a clear objective.

Plan Mode (Shift+Tab twice)

This puts Claude in architect mode — it can observe, read files, and analyze, but it **cannot modify files**. Use it to get Claude's analysis before you let it act. This forces structured, reasoned responses before any code is written.

Use TodoWrite for structured execution

For complex tasks, explicitly instruct Claude to plan:

```
Before writing any code, create a detailed TODO plan with at least
10 items covering all changes needed. Mark each with priority.
Complete items one at a time, running tests after each change.
Mark items complete as you go.
```

The TODO state gets injected as a system message after each tool use, preventing goal drift during long execution loops. This is one of Claude Code's most powerful built-in mechanisms for keeping the agent on track.

The "progressive disclosure" approach

For very large features, don't try to build everything in one session. Break it into phases:

1. Phase 1: Data model and API layer
2. Phase 2: Core business logic
3. Phase 3: UI components
4. Phase 4: Integration tests and polish

Each phase gets its own session with clean context. Write progress to external files between phases so the next session can pick up where you left off.

---

4. Context Management Is the Core Skill

Why this is the #1 skill architecturally

The context window is Claude's entire working memory — everything it knows for this session comes from

what's in that window. It holds system prompt, CLAUDE.md, MCP tool definitions, conversation history, file contents, command outputs, and thinking tokens. When it fills up (around 70% capacity), auto-compaction triggers: older turns get summarized, tool outputs get discarded, and information is lost.

Every token in the context window is processed on every API call. Bloated context doesn't just risk compaction — it actively degrades quality, increases latency, and costs more money.

Start fresh sessions aggressively

Use `/clear` between unrelated tasks. Leftover context from a previous task is noise that degrades quality on the current task. Use `/rename` before clearing so you can `/resume` the session later if needed.

Use `/compact` manually at logical breakpoints

Don't wait for auto-compact. When you finish a logical subtask, run `/compact` yourself. You can even direct the compaction: `/compact Focus on the API changes and test results` to tell Claude what to preserve. This gives you much more control than automatic compaction.

Scope investigations narrowly

"Investigate the authentication system" will cause Claude to read dozens of files, filling context with verbose file contents.

Instead: "Read `src/auth/login.ts` and `src/auth/middleware.ts` and explain how the JWT token validation flow works."

If you need broad exploration, use subagents (see Section 8) — they get their own context window and return only a summary.

Put persistent rules in CLAUDE.md, not conversation

Instructions given early in conversation may be lost during compaction. CLAUDE.md is reloaded fresh and survives any compaction. If something must be remembered across the entire session, it belongs in CLAUDE.md.

Watch MCP server token costs

Every MCP server's tool definitions are loaded into context at session start. Three or four MCP servers can consume significant context before you even begin working. Run `/mcp` to check per-server costs and `/context` to see overall usage. Disconnect servers you don't need for the current task.

Monitor your context budget

Run `/context` regularly to see what's consuming space. Use `/cost` to track token usage. Configure your status line to display context usage continuously so you're never surprised by a compaction.

The "write to file" pattern for long tasks

For multi-phase work, write progress to external files:

- `PROGRESS.md` — what's done, what's pending
- `DECISIONS.md` — architectural decisions made
- `TODO.md` — remaining work items

A fresh session can read these files and resume with full signal quality, avoiding the "summary of a summary" degradation that comes from multiple compactions.

---

5. Write Prompts Like Engineering Requirements

Why precision matters architecturally

The model is a probability engine that follows the strongest signal. Ambiguous prompts produce ambiguous results because the model distributes probability across many plausible interpretations. Specific prompts concentrate probability on the interpretation you want.

Every correction and clarification costs tokens, context space, and risks the correction spiral. Getting it right the first time is dramatically cheaper than iterating.

Be specific about inputs, outputs, and constraints

❌ "Add tests for the auth module"

✅ "Write unit tests for src/auth/login.ts using Jest and React Testing Library. Cover four cases: successful login, invalid credentials, expired token, and network timeout. Follow the patterns in src/auth/signup.test.ts. Mock the API calls using the existing src/test/mocks/api.ts helpers."

Reference files directly

Use tab-completion in your terminal to add exact file paths. Tell Claude which files to read for context and which to modify. Don't make it guess — guessing wastes context on exploration.

Provide external context

- **URLs:** Paste links to docs, GitHub issues, API references — Claude will fetch and read them
- **Images:** Drag and drop screenshots of UI designs, error messages, or architecture diagrams
- **Piped data:** `cat error.log | claude "explain this error and fix it"`
- **Let Claude fetch:** "Use `foo-cli-tool --help` to learn about foo tool, then use it to solve X"

Set explicit constraints

```
Output format: TypeScript with strict types
```

```
Code style: follow existing patterns in src/utils/
Testing: Jest with 80%+ coverage target
Dependencies: don't add new npm packages without asking
Scope: make the minimum change necessary — do not refactor surrounding code
```

Specify non-obvious things the model can't infer

- Which testing framework and assertion library you use
- Whether you want functional components or class components
- Your preferred error handling patterns (Result types? Exceptions? Error boundaries?)
- Import conventions (barrel files? Direct imports? Path aliases?)

These seem obvious to you because you've been in the codebase. Claude is seeing it for the first time every session.

Use the "devil's advocate" pattern before shipping

```
I'm about to deploy this change to production. Play devil's advocate:
- What edge cases did I miss?
- What could fail under load?
- What assumptions might be wrong?
- What happens if external services are down?
- How does this handle bad data?
Be pessimistic. Think like a hacker, not a builder.
```

---

6. Verification Is Non-Optional

Why this is architecturally critical

The same mechanism that enables LLMs to write creative code and generalize to novel problems also enables confident hallucination. Claude will generate code that looks correct, uses real-sounding API methods, follows reasonable patterns — but the methods might not exist or the logic might be subtly wrong. This is fundamental to the technology, not a bug to be fixed.

Your job is to build verification into every workflow so that the loop catches errors automatically.

Build verification into every interaction

1. Have Claude **run tests** after every code change
2. Have Claude **run lint and typecheck** after every code change
3. Have Claude **run the build** to verify compilation

Add this to CLAUDE.md so it happens automatically:

```
## After ANY code change:
1. Run `npm run typecheck`
2. Run `npm run lint -- --fix`
3. Run `npm run test -- --related`
If any fail, fix them before moving on. Never declare a task complete
without all three passing.
```

Use hooks for non-negotiable checks

CLAUDE.md instructions are advisory — the model can ignore them, especially under context pressure. Hooks are **deterministic** and always execute:

```json
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Write(*.ts)",
      "hooks": [{
        "type": "command",
        "command": "npx prettier --write \"$file\""
      }]
    }, {
      "matcher": "Edit(*.py)",
      "hooks": [{
        "type": "command",
        "command": "python -m black \"$file\""
      }]
    }]
  }
}
```

Hooks can also serve as **security gates** with PreToolUse — blocking dangerous operations before they execute, regardless of what the model decided to do.

Git is your safety net

Claude Code creates checkpoints you can roll back to. Always:

- Work on feature branches, not main
- Commit before asking Claude to make large changes
- Use `git diff` to review what Claude actually changed before committing
- If Claude goes off the rails after several iterations, revert to a checkpoint

Review what Claude produces

Don't blindly trust passing tests. Read the code. Check that:

- The logic handles edge cases (not just happy path)
- Error handling is appropriate (not just `catch (e) {}`)
- The implementation matches the spec, not just the test
- No unnecessary dependencies were added
- No security issues were introduced

The "self-test" instruction for extended thinking

For complex algorithmic work, ask Claude to verify its own work:

```
Write a function to [X]. Before declaring it complete, verify your
solution by running through test cases for edge cases in your
extended thinking. Fix any issues you find before outputting.
```

---

7. Model Selection Is an Architectural Decision

Understanding the tradeoffs

Different models have fundamentally different capabilities, costs, latency profiles, and failure modes. Using them deliberately is a core engineering skill.

Claude Opus 4.6 — The most capable model

- Best for: Complex architecture, multi-step reasoning, ambiguous problems, large refactors, planning
- Watch out for: Expensive ($15/M input, $75/M output), strong tendency to over-use subagents, slower, overkill for routine tasks
- Tip: Add to CLAUDE.md: "Use subagents only for parallel, independent workstreams. For simple searches, use Grep/Glob directly."

Claude Sonnet 4.5 — The workhorse

- Best for: Most daily coding, refactoring, test writing, implementation from specs, ~80% cheaper than Opus
- Watch out for: Less reliable on truly complex architectural reasoning
- Tip: This should be your default model for implementation work

Claude Haiku 4.5 — The sprinter

- Best for: Simple tasks, quick lookups, fast iteration, saving tokens
- Watch out for: Degrades significantly faster with many instructions, unreliable for complex planning
- Tip: Don't use for tasks requiring many CLAUDE.md rules

Cost-aware model switching

Switch models mid-session with `/model`. Use Opus for the hard planning and architectural decisions, then switch to Sonnet for implementation. This can reduce costs by 60%+ without sacrificing quality where it matters.

For subagents, you can set a different model to save even more:

```bash
export CLAUDE_CODE_SUBAGENT_MODEL="claude-sonnet-4-5-20250929"
```

This runs your main session on Opus while subagents use Sonnet for focused tasks.

Extended thinking: powerful but expensive

Extended thinking is enabled by default with a budget of ~32K tokens. Thinking tokens are billed as output tokens (the expensive kind). For simple tasks, this is pure waste.

Optimize thinking costs:

- For Opus 4.6: Use the `effort` parameter — low for simple tasks, high for complex ones
- Lower the thinking budget for routine work: `MAX_THINKING_TOKENS=8000`
- Disable thinking entirely for trivial tasks via `/config`
- Don't try to force more thinking tokens for tokens' sake — research shows diminishing returns above 32K

When to use maximum thinking:

- Complex algorithmic problems
- Multi-file architectural decisions
- Debugging subtle, non-obvious bugs
- Security review and threat modeling

Average costs to budget for

The average Claude Code user spends about $6/day, with 90% staying under $12/day. API users on Sonnet average $100-200/month. Opus-heavy workflows cost significantly more. Use `/cost` to track your actual usage.

---

8. Master Subagents for Context and Parallelism

Why subagents matter architecturally

Subagents solve the two biggest constraints of the agentic loop: context window limits and sequential execution. Each subagent gets its own fresh context window (up to 200K tokens) and works independently, returning only a summary to the parent. This means:

- Exploration doesn't pollute your main context
- Multiple tasks can run in parallel
- Each agent can go deep on its specific problem without crowding out others

Built-in subagents

Claude Code includes three built-in agents:

- **Explore** — Read-only, optimized for codebase search and analysis. Claude delegates to this automatically when searching.
- **Plan** — Research agent used during Plan Mode to gather context before presenting a plan.
- **General-purpose (Task)** — Full capabilities including file editing. Used for complex, multi-step operations.

When to use subagents

- The task produces verbose output you don't need in main context
- Multiple independent tasks can run in parallel
- You need deep exploration that would fill your main context
- The work is self-contained and can return a useful summary

When NOT to use subagents

- Simple tasks (a direct Grep is faster than spawning an agent)
- Tasks that need context from the main conversation
- Sequential work where each step depends on the previous
- When you need real-time visibility into what's happening (subagents work in isolation)

The "Master-Clone" vs "Lead-Specialist" debate

There are two schools of thought:

Custom specialist agents (Lead-Specialist): Define agents with specific roles (code-reviewer, security-auditor, frontend-expert) with custom system prompts and tool restrictions. Good for reproducible pipelines.

General Task clones (Master-Clone): Let the main agent spawn copies of itself with your full CLAUDE.md context, deciding dynamically when and how to delegate. More flexible, less configuration overhead.

Both work. Start with Task clones for simplicity. Move to custom agents when you have repeatable workflows that benefit from specialization.

Critical subagent tip: Invocation quality

Most subagent failures are **invocation failures** — the parent agent sends vague instructions. Subagents can't ask clarifying questions. Ensure your prompts to subagents include:

- Specific file paths to examine
- Clear success criteria
- Relevant context the subagent needs to know
- Expected output format

Cost optimization with subagents

Set subagents to use a cheaper model:

```bash
export CLAUDE_CODE_SUBAGENT_MODEL="claude-sonnet-4-5-20250929"
```

For massive parallel work (like refactoring across many files), consider external parallelization:

```bash
# More scalable than agent-managed subagents
claude -p "in /pathA change all refs from foo to bar" &
claude -p "in /pathB change all refs from foo to bar" &
wait
```

---

9. Know the Failure Modes and Their Fixes

The correction spiral

You point out an error, Claude "fixes" it but breaks something else, you correct again, it introduces a third bug. Each correction pollutes context with wrong attempts and confused reasoning.

Fix: After two failed corrections, `/clear` and write a better initial prompt incorporating what you learned. A fresh session with a good prompt almost always beats a polluted session with accumulated confusion.

The infinite exploration

You ask Claude to "investigate" something open-ended. It reads 50+ files, fills the context with verbose content, and produces a vague summary.

Fix: Scope investigations tightly: "Read these three files and explain X." Or delegate to a subagent so the exploration uses its own context window, returning only a summary.

The plausible hallucination

Claude generates code that looks correct — uses real-sounding API methods, follows reasonable patterns — but the methods don't exist or the logic is subtly wrong. Especially dangerous with unfamiliar APIs.

Fix: Always verify with tests, typechecking, and compilation. For unfamiliar APIs, tell Claude to read the actual documentation first: "Fetch the Stripe SDK docs at [URL] before implementing the payment flow." Don't let Claude rely on training data for API details — APIs change.

Context degradation over long sessions

After multiple compactions, Claude is working from a "summary of a summary." Quality degrades, decisions are forgotten, and instructions from early in the session get lost.

Fix: Put persistent rules in CLAUDE.md (survives compaction). Write progress to external files. For long tasks, prefer the "clear and resume from files" pattern over relying on compaction: save state to `PROGRESS.md`, `/clear`, start fresh session reading the progress file.

The over-eager refactor

You ask for a small bug fix. Claude rewrites the entire module "to be cleaner" while also adding "improvements" you didn't ask for.

Fix: Be explicit about scope: "Make the minimum change necessary to fix this bug. Do not refactor surrounding code. Do not change any code not directly related to the fix."

Scope creep and unsolicited changes

Claude runs git commands when you asked it to read files, or installs packages you didn't ask for, or modifies files outside the scope of the task.

Fix: Use permission rules to constrain the action space:

```json
{
  "permissions": {
    "deny": ["Bash(git push *)", "Bash(rm -rf *)", "Bash(sudo *)"],
    "ask": ["Bash(git commit *)", "Bash(npm install *)"]
  }
}
```

Claims completion without verification

Claude says "Done!" but didn't actually run the tests or verify the changes work.

Fix: Add to CLAUDE.md: "Never declare a task complete without running tests and typecheck. Show me the passing output."

The stuck-in-a-loop failure

Claude keeps reading the same files, re-reading after compaction, or retrying the same failing approach.

Fix: Interrupt (Escape or Ctrl+C), steer to a different approach, or `/clear` and start over with a more specific prompt.

---

10. Leverage the Full Extension System

Custom slash commands

Create reusable workflows in `.claude/commands/`:

```markdown
<!-- .claude/commands/test.md -->
Write comprehensive tests for $ARGUMENTS following our testing
patterns in src/__tests__/examples/. Use Jest + RTL. Cover happy
path, error cases, and edge cases. Run tests after writing.
```

Then `/test MyComponent` executes the same reliable workflow every time.

Important opinion from practitioners: Don't overdo slash commands. If you have a long list of complex custom commands that engineers must learn, you've created an anti-pattern. The point of an agent is natural language input. Use commands as simple personal shortcuts, not as a replacement for a good CLAUDE.md.

Skills (on-demand knowledge)

Skills are markdown-based guides that Claude loads only when relevant. Unlike CLAUDE.md (loaded every session), skills load on demand, saving context.

Use skills for:

- Specialized knowledge not needed in every session
- Methodology guides (how to create a database migration, how to write API docs)
- Domain-specific instructions for infrequent tasks

Set `disable-model-invocation: true` for skills you only invoke manually, keeping their descriptions out of context until needed.

MCP servers (external integrations)

Connect to GitHub, Jira, Slack, Notion, Figma, databases, and more:

```bash
claude mcp add github-server  # Add GitHub integration
```

Powerful for workflows like:

- "Read the open GitHub issues and fix the highest priority one"
- "Update the Jira ticket when this PR is ready"
- "Check Slack for the latest deployment status"

Cost awareness: Every MCP server adds tool definitions to every request. Three or four servers can consume significant context before you start working.

Hooks for deterministic behavior

Hooks are the guarantee layer. Unlike CLAUDE.md rules (advisory), hooks always execute:

- **PreToolUse:** Security gates, validation, blocking dangerous operations
- **PostToolUse:** Auto-formatting, linting, logging, external notifications
- **Lifecycle hooks:** Run at session start/end for setup/teardown

Example pipeline with hooks:

- Edit `.ts` file → PostToolUse runs Prettier → PostToolUse runs ESLint → catches issues immediately

GitHub App integration

Run `/install-github-app` to have Claude automatically review your PRs. Especially valuable as AI-assisted coding increases PR volume — the AI that helped write the code can also help review it.

---

11. Cost Management and Token Discipline

Understanding the cost model

Token costs scale with context size — more context = more tokens processed per message. Claude Code optimizes via prompt caching (repeated content like system prompts) and auto-compaction, but you control the biggest cost drivers.

Practical cost reduction strategies

Clear between tasks. Stale context wastes tokens on every subsequent message. This is the single easiest cost saving.

Use the right model. Sonnet for routine work, Opus for hard problems, Haiku for simple tasks. Don't use a $75/M output model for a task a $15/M model handles fine.

Control extended thinking. Thinking tokens are billed as output tokens. For Opus 4.6, use the `effort` parameter: low for simple tasks, high for complex reasoning. Disable thinking entirely for trivial lookups.

Lean CLAUDE.md. Every extra line is reprocessed with every message. Move specialized instructions to Skills.

Preprocess with hooks. Instead of Claude reading a 10,000-line log file, use a hook to `grep ERROR` and feed only matching lines. Reduces context from tens of thousands of tokens to hundreds.

Use code intelligence plugins. Language servers give Claude precise symbol navigation instead of text-based search, reducing file reads and context consumption.

Set automation safeguards. For CI/CD pipelines, configure `max_turns` and `timeout_minutes` to prevent runaway costs.

Budgeting expectations

- Average: ~$6/developer/day
- 90th percentile: ~$12/developer/day
- Monthly with Sonnet: ~$100-200/developer
- Monthly with heavy Opus: significantly more
- Use `/cost` to track actual usage

---

12. Working With Your Team

Version-control your Claude Code configuration

- `CLAUDE.md` — commit to repo, shared with team
- `.claude/settings.json` — commit to repo (shared permissions, hooks)
- `.claude/settings.local.json` — git-ignored (personal preferences)
- `.claude/commands/` — commit to repo (shared workflows)
- `.claude/agents/` — commit to repo (shared subagent definitions)

Establish team conventions

- Which CLAUDE.md rules are mandatory vs. recommended
- Required hooks (formatting, linting)
- Permission baselines (what's auto-allowed, what requires approval)
- Model selection guidelines (when to use Opus vs. Sonnet)
- Session hygiene practices (when to `/clear`, when to `/compact`)

Progressive trust escalation

Start new team members with conservative permissions (ask for everything). As trust builds, allow more auto-approved tools. Use `.claude/settings.json` to set the team baseline and let individuals customize via `settings.local.json`.

---

13. Security Considerations

Permission discipline

The default "ask for everything" mode exists for good reason. `--dangerously-skip-permissions` is convenient but carries real risk — especially with subagents, which inherit the permission mode.

Best practice: Define explicit allow/deny rules rather than blanket skip:

```json
{
  "permissions": {
    "allow": ["Read", "Edit", "Bash(npm run *)", "Bash(git status)", "Bash(git diff *)"],
    "deny": ["Bash(rm -rf *)", "Bash(sudo *)", "Bash(curl *)"],
    "ask": ["Bash(git commit *)", "Bash(git push *)", "Write"]
  }
}
```

Sensitive file protection

```json
{
  "permissions": {
    "deny": ["Read(.env*)", "Read(./secrets/**)", "Write(production.config.*)"]
  }
}
```

Subagent security

When using `bypassPermissions`, all subagents inherit this mode. Subagents may have different system prompts and less constrained behavior. Be especially careful with custom subagents that have broad tool access.

---

14. The Meta-Skill: Think Like the Architecture

Everything in this guide follows from understanding how the system works:

Architecture Fact	Why It Matters	What To Do
Model is stateless	Knows nothing between sessions	Invest in CLAUDE.md
Context window is finite	Degrades when full	Manage context aggressively
Agentic loop compounds errors	Each step risks failure	Plan first, minimize steps
LLMs are confident when wrong	Plausible hallucinations	Always verify with tests
Compaction loses information	Summaries degrade	Use fresh sessions, write to files
Probability follows strongest signal	Ambiguity produces ambiguity	Be specific in prompts
Thinking tokens cost money	Extended thinking is expensive	Right-size thinking to task
Tools execute locally	Real actions on your machine	Set permission boundaries
Subagents have isolated context	Exploration doesn't pollute	Use subagents for broad searches

The fundamental shift: You're no longer a developer who writes code. You're an **engineering lead who directs, constrains, and verifies an autonomous agent**. Your value is in knowing *what* to build, *how* it should be architected, *what quality bar* to hold, and *when to trust vs. verify* the agent's work.

The code is generated. The engineering is still yours.